
Verification Specification

Processor Core for the RV32I
(RISC-V 32-Bit Integer Base) Instruction Set

Verification environment, methodology and plan

Document	Verification Specification — RV32I Processor Core
Document type	Verification environment specification
Subject	RV32I processor core (three-stage pipeline)
Revision	1.2
Date	16 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	2
1.1	Purpose of the document	2
1.2	Validity of the document	2
1.3	Definitions of terms and abbreviations	2
1.4	Requirements and implementation decisions	3
1.5	Relationship with other documents	3
1.6	Notes on the notation used	3
2	Verification methodology	4
2.1	Overview	4
2.2	Implementation strategy	4
2.3	Verification plans	5
2.4	Reference model	5
3	Verification environment	6
3.1	Overview	6
3.2	Control and drivers	6
3.2.1	Clock and reset generation	6
3.2.2	Instruction supply	6
3.2.3	Bus slave models	7
3.2.4	Interrupt stimulus	7
3.3	Monitors and checkers	7
3.3.1	Bus protocol monitors	7
3.3.2	Assertion layer	8
3.3.3	Self-checking comparison	8
3.3.4	Functional coverage	9
3.4	Simulation environment setup	9
3.4.1	Structure	9
3.4.2	Scripts	9
3.4.3	Setup and configuration	10
4	Annex	10
4.1	Verification plan	10
4.2	Measured results	15

1 Introduction

1.1 Purpose of the document

This document specifies the verification environment applied to the RV32I processor core, the methodology behind it, and what that environment establishes about the design.

The verification plan in the annex lists what is checked, and where each check is applied.

1.2 Validity of the document

The document is valid for the verification environment that targets the processor core described in the Design Specification. Results reported here were measured on the current revision, on both simulators.

The regression contains further runs that target other blocks. Every figure reported here is measured over the runs that exercise the core, and excludes those other runs.

1.3 Definitions of terms and abbreviations

Term	Definition
AMBA	Advanced Microcontroller Bus Architecture. The ARM bus interface family that AXI4-Lite belongs to.
Assertion	A property stated in a temporal language and evaluated on every clock cycle of every simulation in which it is active.
AXI4-Lite	The register-access subset of the AMBA Advanced eXtensible Interface, revision 4.
Backpressure	Withholding a ready signal to delay a transfer, used here to vary bus timing.
Block-level test	A test that drives one design block on its own ports, independently of any program running on the core.
Checker	Logic that compares observed behaviour against expected behaviour and records a mismatch.
CSR	Control and Status Register.
Directed test	A test with hand-chosen stimulus aimed at a specific behaviour, as opposed to randomly generated stimulus.
Functional coverage	A set of named scenarios, each counted when it occurs, used to show which planned situations a run actually reached.
ISA	Instruction Set Architecture.
Monitor	A passive observer of an interface that reports protocol violations without driving anything.
Reference model	An independent implementation of the expected behaviour, against which the design's output is compared automatically.
Regression	The complete set of runs, executed together from one compilation to confirm that previously working behaviour still works.
RV32I	The 32-bit integer base instruction set of the RISC-V ISA.
Scoreboard	A record of expected results accumulated during a run and compared at its end.
Self-checking test	A test that contains its own expected results and reports a verdict, rather than producing output for a human to inspect.
System-level test	A test in which real software executes on the core against behavioural memories.
WARL	Write Any Values, Reads Legal Values.

Term	Definition
WFI	Wait For Interrupt.

1.4 Requirements and implementation decisions

The original Siemens briefs are kept in `cpu/docs/brief/`: *RV32I 3-Stage Pipeline CPU* and *Programmable Interrupt Controller with Advanced Scheduling*. They define the project scope. Instruction and CSR semantics are checked against the cited RISC-V manuals, and bus handshakes against the AXI specification. These authored specifications describe the implementation; they are not independent evidence that the implementation is correct.

Details absent from the briefs, including the register map, bus error policy and the sixteen-source claim/EOI interface, are project decisions. The CPU brief’s eight-source interface and the PIC brief’s sixteen-source interface still require a common interface agreement; the implemented contract is stated explicitly in the design specifications.

1.5 Relationship with other documents

Document	Relationship
Design Specification — RV32I Processor Core	Specifies the design this environment verifies. Every behaviour referred to here is defined there; this document does not restate architectural detail. Section numbers of that document are cited where a verification item maps directly onto a specified behaviour.
The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, version 20191213	Source of the expected instruction behaviour used to derive directed stimulus.
The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, version 20211203	Source of the expected trap, CSR and interrupt behaviour used to derive directed stimulus.
AMBA AXI and ACE Protocol Specification, ARM IHI 0022E	Source of the protocol rules checked by the bus monitors and the bus assertions.

1.6 Notes on the notation used

- Fixed-width type denotes a signal, a register or a literal value.
- Italic type marks emphasis; bold type marks a run-in heading.
- Verification plan items are identified as **VP-*nn*** and are listed in the annex.
- “The core” is the design under verification; “the controller” is the interrupt controller instantiated with it.

2 Verification methodology

2.1 Overview

Directed tests, independent reference models, protocol monitors, assertions and functional coverage check complementary aspects of the design.

Table 2: Verification mechanisms and their reach

Mechanism	What it establishes, and what it cannot
Directed self-checking tests	Establish that a specific scenario produces a specific result. Each test carries its own expected values and prints a verdict, and the runner requires explicit completion with zero mismatches. They see only the scenarios someone thought to write.
Protocol monitors	Establish that both bus interfaces obey the protocol, continuously and independently of what the design and the memory models believe is happening. They see protocol violations, not wrong results.
Assertions	Establish invariants that must hold in every cycle of every run in which they are active, including cycles no test was written for. They see rule violations, not wrong values.
Functional coverage	Establishes which planned scenarios a run actually reached. It proves nothing about correctness; it shows where the directed tests are silent.

2.2 Implementation strategy

Verification is applied at system and block level.

System level. The core executes a real program, assembled from source into an instruction image, against behavioural memory models. The program is the checker: it computes results, stores them where the environment can read them, and drives the sequence of events. This is the closest approximation to how the core is used, and it is the level at which the interaction between fetch, execution, the memory interface and the trap mechanism is exercised as a whole. A second system-level test instantiates two cores against one shared memory through an arbiter.

Block level. Individual design blocks are driven directly on their own ports. This level exists for properties a running program cannot isolate:

- reset values, which a program-driven test has already destroyed by the time it observes anything;
- behaviour at operand boundaries that a hand-written program does not naturally produce;
- each trap cause on its own, so that a single wrong cause is a failing test rather than one line among many;
- structural properties such as full index coverage of the predictor arrays, and the boundaries of the return address stack;
- parameter settings that no core-level run instantiates.

Bus timing variation. The single-core system test is repeated under four memory timing configurations: nominal, high fixed latency, and two seeds of randomised ready-signal backpressure. A design that only works when every slave answers immediately has not been verified. The mechanisms that hold state across a response — the outstanding-transaction tracking in the fetch stage and the per-channel handshake tracking in the load/store unit — are precisely the ones that fail only when a slave takes its time.

Two simulators. The regression runs on the primary simulator. A second, independent simulator additionally compiles and runs the assertion layer and the functional coverage model,

which the primary simulator’s available edition cannot. Both flows are run for every revision, so an assertion is live evidence rather than a description. Compiling the same sources with two front ends is itself a check, since each tolerates different oversights.

Pass criteria. A run passes only if all of the following hold: every self-check matched, no protocol monitor recorded a violation, no assertion fired, and every required functional coverage bin was reached. A missed required coverage bin fails the run in the same way a mismatched value does, because a scenario the plan claims and the stimulus never reaches is a hole in the plan, not a detail.

2.3 Verification plans

The verification plan is organised by design area rather than by test, so that a plan item can be traced to the behaviour it covers regardless of which test happens to implement it. Each item names the behaviour, how it is stimulated, how the expected result is determined, and where the check is applied. The items are listed in the annex.

Items are grouped into six areas:

Table 3: Verification plan areas

Area	Covers
Instruction execution	Instruction decode, the arithmetic path, operand forwarding, control transfers and branch prediction.
Memory interface	The two bus ports, byte lanes, alignment and error responses.
Traps and exceptions	Cause selection, recorded state, precision, and the vector mode.
Interrupts	Acceptance conditions, the claim and end-of-interrupt handshake, nesting, masking, and wait-for-interrupt.
Control and status registers	Access classes, field behaviour, reset values and the counters.
System properties	Operation of more than one instance, and sustained throughput.

2.4 Reference model

`isa_reference.py` encodes instructions and interprets their architectural effects independently of both the RTL and `asm.py`. It produces an instruction image, an expected PC/destination/value trace and a final data-memory image. `rv32i_tb_isa` compares each completed instruction against the trace and then compares all 256 data words. Expected results do not depend on the decoder outputs or the RTL ALU function.

The deterministic program executes 1 287 instructions and reaches 50 named generator categories. It includes arithmetic boundaries, all shift amounts, signed and unsigned comparisons, each branch condition taken and not taken, backward branches, JAL/JALR, LUI/AUIPC, FENCE, all load/store widths and byte lanes, and CSR register/immediate forms. Loads after each partial store check untouched lanes before another store can overwrite the evidence. The model also checks `minstret` ordering around CSR reads and writes. Four ModelSim configurations vary latency and ready-signal backpressure.

`rv32i_tb_counters` drives the CSR interface directly. It checks all seven implemented 64-bit counters, both writable halves, increment carry, write priority, CSRCS/CSRRC masks, pure reads and the five fixed event identifiers. The expected arithmetic follows Zicsr and the RV32 counter-half access rules.

Existing directed tests retain precomputed architectural constants and protocol checks. Their results complement the trace; they do not replace it. The reference program is finite and does not model every trap interleaving or all possible operands. No formal proof or external ISA compliance certification is claimed.

Testing the checks. The mutation script copies sources into a temporary workspace, runs passing baselines, injects one defect at a time and requires a behavioural failure in the selected test. A compile error does not count as detection. All 16 injected defects were detected across CPU, PIC and SoC. CPU mutations cover subtraction, arithmetic shift, load sign extension, byte/halfword strobes and counter write/carry priority. A surviving byte-strobe mutation exposed a missing untouched-lane check; the trace was strengthened and both partial-store mutations then failed.

3 Verification environment

3.1 Overview

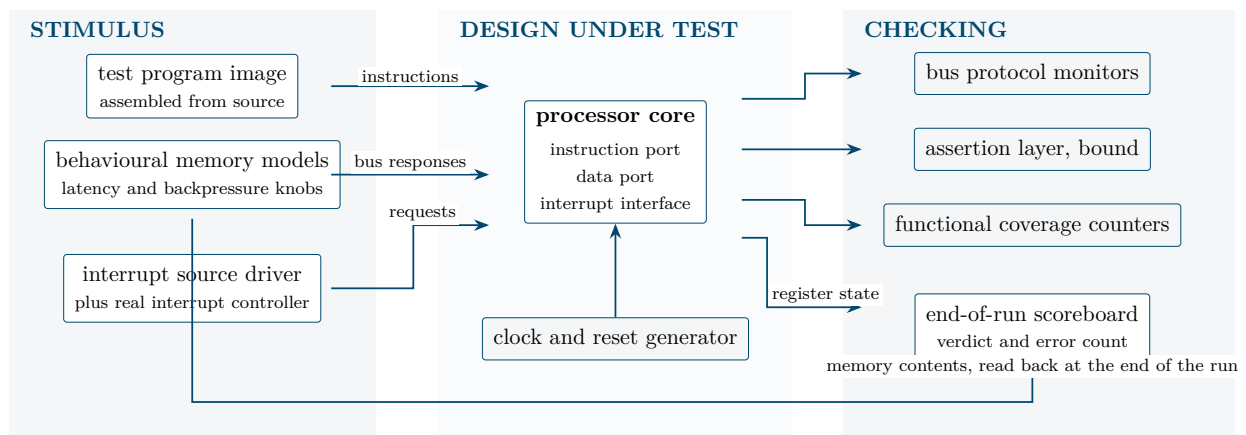


Figure 1: System-level verification environment. Stimulus enters from the left: the assembled program supplies instructions, the behavioural memories answer both bus ports with configurable timing, and the interrupt path is driven through a real interrupt controller so the core sees realistic request sequencing. On the right, three independent observers watch the core — passive protocol monitors on each bus, an assertion layer bound into the design, and functional coverage counters — while the end-of-run scoreboard compares the memory contents and the register state left behind by the program against precomputed values. A run fails if any observer or the scoreboard reports a mismatch.

At block level the structure is simpler: the block is instantiated on its own, a directed driver applies stimulus to its ports, and the same self-checking comparison mechanism records the verdicts. Register-port tests also use the shared bus monitor. The Verilator flow binds applicable assertions into both block-level and system-level tests; user functional coverage is collected in the nominal CPU system test.

3.2 Control and drivers

3.2.1 Clock and reset generation

A single generator produces the clock and the asynchronous, active-low reset for every run. Reset release is the starting point of every test, which is what allows the block-level tests to check reset values before any other activity.

3.2.2 Instruction supply

The system-level stimulus is a program written in assembly and translated into an instruction image loaded by the instruction memory model. The translation step also emits the address of every label in the program, and the environment uses those addresses in its checks. This matters for maintainability: a change to the program that shifts a label updates the corresponding expected value automatically, instead of silently invalidating a hand-written address constant.

The program is written to exercise every instruction group, every synchronous trap cause, the CSR access rules and the interrupt sequence, and to leave its results in architectural registers and in a region of data memory that the environment reads at the end of the run.

3.2.3 Bus slave models

Both bus ports are answered by a behavioural AXI4-Lite slave model with internal memory. Each model decodes its own address range and answers an access outside that range with a decode error, which is how access-fault behaviour is stimulated without an interconnect.

The models carry the timing knobs that make the bus timing sweep possible:

Table 4: Bus model configuration

Knob	Effect
Fixed read latency	Wait cycles inserted before read data is presented.
Fixed write latency	Wait cycles inserted before the write response is presented.
Backpressure rate	Probability per cycle that a ready signal is withheld, plus a random number of extra response wait states.
Seed	Makes a randomised configuration reproducible; the same seed replays the same run exactly.

3.2.4 Interrupt stimulus

The core’s interrupt interface is not driven directly with hand-made request patterns. Instead the real interrupt controller is instantiated between the environment and the core, and the environment plays the peripheral sources the way real peripherals would: a source is raised on an event and held until the handler clears it. This gives the core realistic request sequencing, including requests that remain asserted through their own handler.

The source movements are chosen to create specific situations rather than to be plausible: one source is enabled in the controller but never in the core, so that it must never be accepted; two sources rise together so that priority order is observable; and one source rises while a data transaction is in flight, so that the acceptance boundary is exercised.

At block level, interrupts are applied directly to the core’s interface, which is what allows individual source numbers to be selected deterministically.

3.3 Monitors and checkers

3.3.1 Bus protocol monitors

One passive monitor instance observes each bus. Monitors drive nothing; they observe the same signals the design and the slave models exchange and flag a violation in the cycle it occurs. Checked rules:

- **Stability.** Once a valid signal is asserted it remains asserted, and its payload remains unchanged, until the handshake completes. Applied to all five channels.
- **Ordering.** Read data appears only with a read outstanding; a write response appears only after both the address and the data handshakes have completed; a second request on a channel before the response of the first violates the single-outstanding rule.
- **Defined values.** No undefined value on a valid or ready signal out of reset, and none on a payload while its valid signal is asserted.
- **Response legality.** The exclusive-access response encoding is not legal on AXI4-Lite and is flagged.

Each monitor exposes an error count that must be zero at the end of the run, and read and write transaction counts. The counts are checked to be non-zero, which prevents a silent bus from being mistaken for a clean one.

3.3.2 Assertion layer

Assertions are bound into the design rather than written into it, so no design source is modified for verification. Two groups exist.

Table 5: Assertion groups

Group	Properties
Bus protocol	The same stability, ordering, outstanding-transaction and response-legality rules the monitors apply, expressed as temporal properties and bound to each port. The instruction port additionally asserts that every address is word aligned, and that no request is issued while reset is asserted.
Core invariants	The claim and end-of-interrupt outputs are each exactly one cycle wide; a claim never occurs without the trap-active output; an end-of-interrupt only follows a return instruction; an interrupt is never accepted while a data transaction is active; trap entry always raises the trap-active output, a return at the last open level always drops it and a return at an inner level always leaves it raised; a trapping instruction never reaches writeback; a stalled execute stage always feeds a bubble to writeback; register zero reads zero on both read ports; the byte strobe pattern is always one of the seven legal shapes; a read and a write are never issued together on the data port; and no new instruction fetch is issued while the core is sleeping in wait-for-interrupt.

ModelSim runs all functional tests and ordinary protocol monitors. Verilator runs all fifteen distinct CPU/PIC testbenches with the applicable bound assertions; it also evaluates functional coverage in the nominal system test. An assertion contributes evidence only in instances and scenarios actually executed by that flow.

One property in this group was wrong until the trap-active output was exercised under nesting. It required every return to drop the output, which holds only when a single trap level is open; a return closing an inner level has to leave it raised. The behaviour stated above — a return at the last open level drops it — is what the core has always done, and the property now says so, split into the two cases so that both directions are checked.

3.3.3 Self-checking comparison

The shared comparison treats undefined values as mismatches. Failures are always printed; successful individual checks are printed only with `+verbose`. Each test calls `finish_test`, which requires zero errors and sets `test_done` before completing. Watchdogs fail rather than ending silently.

The ModelSim runner checks completion, zero errors and each requested parameter value after elaboration. It returns a nonzero exit code on failure. The GUI additionally requires the exact set of completion banners and the regression completion marker. The Verilator runner rejects nonzero process status, failure messages and missing completion or required coverage markers. Banner counts alone are insufficient evidence.

3.3.4 Functional coverage

The coverage model counts named scenarios in the cycle they occur, sampled at four points: instruction commit, trap entry, control transfer resolution, and bus response handshakes. Counted scenarios include the instruction classes, the load and store widths, each branch condition in both directions, the four combinations of prediction against outcome, each trap cause, the CSR operation forms, the forwarding combinations, the wait-for-interrupt scenarios, the return stack push and pop cases, the interrupt claims per source, and the bus response codes including error responses and backpressure.

Each bin is marked required or optional. A required bin that is never reached fails the run. The optional bins are those that a nominal run legitimately does not reach: the negative cases that must never occur, and the bins that only the randomised-backpressure configurations produce.

The coverage report is produced by the secondary simulator flow. On the run measured for this revision, 88 of 92 bins were reached. The four that were not are all optional and all of them are bins the nominal configuration cannot produce: two bins for source 5 (trap and claim), deliberately masked throughout the run, and the instruction/data AR backpressure bins. Backpressure is checked in the ModelSim timing sweeps; this nominal coverage total does not measure those additional runs. No required bin was missed, so the coverage gate passed.

3.4 Simulation environment setup

3.4.1 Structure

The environment is organised in four layers, compiled bottom-up in dependency order:

1. the design, ordered from leaf blocks up to the core;
2. the shared verification models — clock and reset generation, the bus slave model, the protocol monitor, and a minimal address decoder and arbiter used to build small bus topologies;
3. the system-level tests, which instantiate the core together with those models;
4. the block-level tests, each of which instantiates one design block.

Interface wiring for the bus bundles is factored into shared definitions so that a port list appears once rather than once per test, and the self-checking comparison is likewise shared.

The compile order is not strictly required by the simulator, which resolves module instances at elaboration. It is maintained because macro and include visibility does depend on order, because a missing block is then reported at the level that needed it, and because downstream tools do require definition before use.

3.4.2 Scripts

Three entry points exist, each doing one thing:

Table 6: Environment entry points

Entry point	Function
Program assembly	Translates the test program sources into instruction images and emits the label address definitions used by the checks. Must run before any system-level test.
Regression	One compilation followed by every run in sequence, each configured through elaboration-time parameter overrides and each ending in its own verdict banner.
Assertion and coverage run	Builds the same tests on the secondary simulator with the assertion layer and applicable assertions included, runs all fifteen benches and the nominal system coverage model, and applies the pass criteria of Section 2.2, including the coverage gate.

3.4.3 Setup and configuration

Per-run configuration is applied at elaboration through parameter overrides on the bus models — read latency, write latency, backpressure rate and seed. Each configured run echoes the values read back from the elaborated design before it starts, so the log itself is evidence that the intended configuration was applied rather than silently ignored.

The prerequisites for running the environment are the two simulators, and a Python interpreter for the program assembly step. No other tool is required and no license-managed tool is needed for the assertion flow, which is why the assertion and coverage layer can also run unattended.

4 Annex

4.1 Verification plan

Every item below is checked, and passed in the regression measured for this revision.

Table 7: Verification plan

ID	Title	Description	Status	Test, checker
Instruction execution				
VP-01	Instruction group execution	Every implemented instruction group executes and produces the architectural result. Results are chained so a lost value propagates.	Passing	System test, register and memory scoreboard
VP-02	Arithmetic operation decode	The mapping from operation class and function fields to the concrete operation, exhaustive over the encodings the decoder can produce.	Passing	Arithmetic block test, per-case comparison
VP-03	Arithmetic boundary values	Add and subtract at the sign boundary, all shift amounts for all three shifts, signed against unsigned comparison, and the bitwise operations over a boundary-pattern matrix.	Passing	Arithmetic block test, per-case comparison
VP-04	Operand forwarding	A result produced in writeback is used by the immediately following instruction, for each source operand and for both together.	Passing	System test, chained results; coverage counters

ID	Title	Description	Status	Test, checker
VP-05	Branch direction and target	Each branch condition taken and not taken, and both jump forms including the link value.	Passing	System test, register and memory scoreboard
VP-06	Predictor learning	A control transfer records its outcome and target and predicts accordingly on the next encounter, in both directions.	Passing	Predictor block test, port-level comparison
VP-07	Predictor indexing and tagging	All entries individually addressable; two addresses sharing an index evict each other rather than producing a false hit.	Passing	Predictor block test, exhaustive over the index
VP-08	Return stack behaviour	Push and pop ordering, routing of a return-tagged entry, fall-back when the stack is empty, overflow beyond the configured depth, and the call-and-return instruction that replaces the top of stack.	Passing	Predictor block test, port-level comparison
VP-09	Return stack disabled	With the stack depth parameter set to zero, a return-tagged entry predicts from the stored target and no output is undefined.	Passing	Predictor block test, second instance
VP-10	Predictor reset	All entries invalid after reset, no undefined value on any output, and return to that state from a fully trained predictor.	Passing	Predictor block test, port and array inspection
VP-11	Illegal instruction has no effect	An illegal encoding writes no register and issues no bus transaction.	Passing	System test and trap test, memory scoreboard
Memory interface				
VP-12	Load and store widths	All five load forms and all three store forms, including sign and zero extension and byte lane selection.	Passing	System test, register and memory scoreboard
VP-13	Byte strobe shapes	Byte and half-word stores drive only the lanes they are supposed to, and every write presents one of the seven legal strobe patterns.	Passing	System test, memory scoreboard per lane; assertion in the secondary flow
VP-14	Store-to-load ordering	A value stored and immediately reloaded returns what was stored.	Passing	System test, memory scoreboard
VP-15	Bus protocol compliance	Handshake stability, channel ordering, single outstanding transaction, and response legality on both ports.	Passing	Protocol monitors, every run; assertions in the secondary flow

ID	Title	Description	Status	Test, checker
VP-16	Bus timing variation	All system-level behaviour repeated under high fixed latency and under two seeds of randomised backpressure.	Passing	System test, four configurations
VP-17	Single outstanding transaction	Neither port issues a second request before the response of the first, and the data port never issues a read and a write together.	Passing	Protocol monitors; assertion in the secondary flow
VP-18	Instruction bus quiet during sleep	No new fetch is issued while the core sleeps in wait-for-interrupt.	Passing	System test measurement; assertion in the secondary flow
Traps and exceptions				
VP-19	Each synchronous cause	One isolated test per cause, checking the recorded cause, return address, associated value and the handler address actually entered.	Passing	Trap test, one case per cause
VP-20	Cause reachability and count	All nine synchronous causes reached within one program, with the number of handler entries counted exactly so a missing or duplicated trap cannot cancel out.	Passing	System test, counter in memory
VP-21	Trap precision	A trapping instruction never reaches writeback.	Passing	Assertion (secondary flow); negative checks in the trap test
VP-22	Misaligned access issues nothing	A misaligned load or store produces no bus transaction.	Passing	Trap test, bus observation
VP-23	Vectored trap entry	In vectored mode an interrupt enters at the base plus four times its cause, while an exception still enters at the base.	Passing	Trap test and system test
VP-24	Return address by trap kind	An exception records the offending instruction; an interrupt records the instruction to resume from; an interrupt that ends a sleep records the following instruction.	Passing	System test and trap test, recorded value compared
Interrupts				
VP-25	Acceptance at an instruction boundary	No interrupt is accepted while a data transaction is in flight; a request raised during a transaction is held and accepted afterwards, with the transaction result intact.	Passing	System test, event ordering; assertion in the secondary flow

ID	Title	Description	Status	Test, checker
VP-26	Claim and end-of-interrupt shape	Both pulses are exactly one cycle wide, the claim is accompanied by the trap-active output, and an end-of-interrupt only follows a return.	Passing	System test invariants; assertions in the secondary flow
VP-27	Return from exception emits no end-of-interrupt	A synchronous exception taken inside an interrupt handler returns without releasing the interrupt level, and the trap-active output stays asserted across it.	Passing	Trap test, end-of-interrupt counter sampled inside the handler
VP-28	Two masks compose	A source enabled in the controller but masked in the core is never accepted, and does not prevent other sources from being served.	Passing	System test, claim counters per source
VP-29	Individual source numbers	Interrupts on the lowest, a middle and the highest source number are accepted and report the correct cause.	Passing	Trap test, three isolated cases
VP-30	Wait-for-interrupt	Sleep entry, wake by an enabled pending interrupt, and fall-through when the global enable is clear.	Passing	System test, recorded values and sleep duration
VP-31	Two nested interrupt handlers	A second interrupt preempts a handler that has re-enabled interrupts; each level is claimed once and each return releases exactly its own level.	Passing	Trap test, claim and end-of-interrupt counters
Control and status registers				
VP-32	Reset values	The reset value of every implemented register.	Passing	CSR block test, read after reset
VP-33	Read-only enforcement	A write to each read-only register traps and leaves the value unchanged, for all three CSR operations.	Passing	CSR block test, per-register
VP-34	Read-only fields	Reserved fields inside writable registers read zero and ignore writes.	Passing	CSR block test, per-field
VP-35	WARL behaviour	Writes to the identification register and to the trap vector mode are accepted without trapping, and only a legal value is read back.	Passing	CSR block test
VP-36	Tied-off registers	The unimplemented counters and event selectors read zero and swallow writes without trapping, across all their addresses.	Passing	CSR block test, all addresses
VP-37	Absent registers	A read or write of an unimplemented address raises an illegal instruction exception.	Passing	CSR block test

ID	Title	Description	Status	Test, checker
VP-38	Pure read exemption	A set or clear operation with a zero source does not count as a write and does not trap on a read-only register.	Passing	CSR block test
VP-39	Hart identifier	The identifier register follows its parameter, so two instances are distinguishable.	Passing	CSR block test with a non-zero parameter; dual-instance test
VP-40	Counter increment	Each counted event increments its counter. Architecturally exact where the count is deterministic; non-zero where it depends on bus timing.	Passing	System test, values read by the program
VP-41	Counter write behaviour	An explicit write uses the old addressed half, preserves the other half and suppresses the implicit increment. Pure reads still count.	Passing	Counter block test and independent instruction trace
System properties				
VP-42	Multi-instance operation	Two instances with distinct identifiers make progress through one arbitrated shared memory and complete a handshake through it.	Passing	Dual-instance test, result and flag comparison
VP-43	Sustained throughput	One instruction per cycle on straight-line code with an instruction memory that answers immediately. Measured as 33 cycles for a window of 33 instructions.	Passing	System test, cycle window measured by the program itself
VP-44	Independent instruction trace	Completed PC, destination, result and final memory match an independent ISA model under four bus timing configurations. Partial stores preserve other lanes.	Passing	ISA reference bench
VP-45	Checker sensitivity	Passing baselines are followed by isolated behavioural mutations. Arithmetic, load/store and counter faults must be rejected by the selected tests.	Passing	Mutation script and detecting benches

4.2 Measured results

The values below were measured on a regression run of the design revision this document describes.

Table 8: Regression result for the runs that exercise the core

Run	Configurations	Checks	Result
System test, nominal timing	1	97	pass
System test, high fixed latency	1	95	pass
System test, randomised backpressure	2	190	pass
Dual-instance test	1	5	pass
CSR access, fields and reset	1	293	pass
Trap causes, one case each	1	120	pass
Branch predictor	1	68	pass
Arithmetic path	1	57	pass
Counter writes and carry	1	159	pass
Total, directed checks	10	1084	0 failures

Four additional ISA trace runs each compare 1 287 completed instructions and 256 data words. They are reported separately from the labelled directed checks in the table.

The complete ModelSim CPU/PIC regression comprises 21 runs and 8 473 labelled checks, plus those four traces and memory comparisons, with no failures. Compilation reports zero errors and zero warnings. The Verilator flow executes all fifteen distinct benches, with no assertion failure; the nominal system coverage result is 88/92 bins, with every required bin reached. These finite results establish the scenarios exercised, not exhaustive correctness.